

Synchronization

- Review threads
- Background
- The Critical-Section Problem
- Java Synchronization
- Semaphores

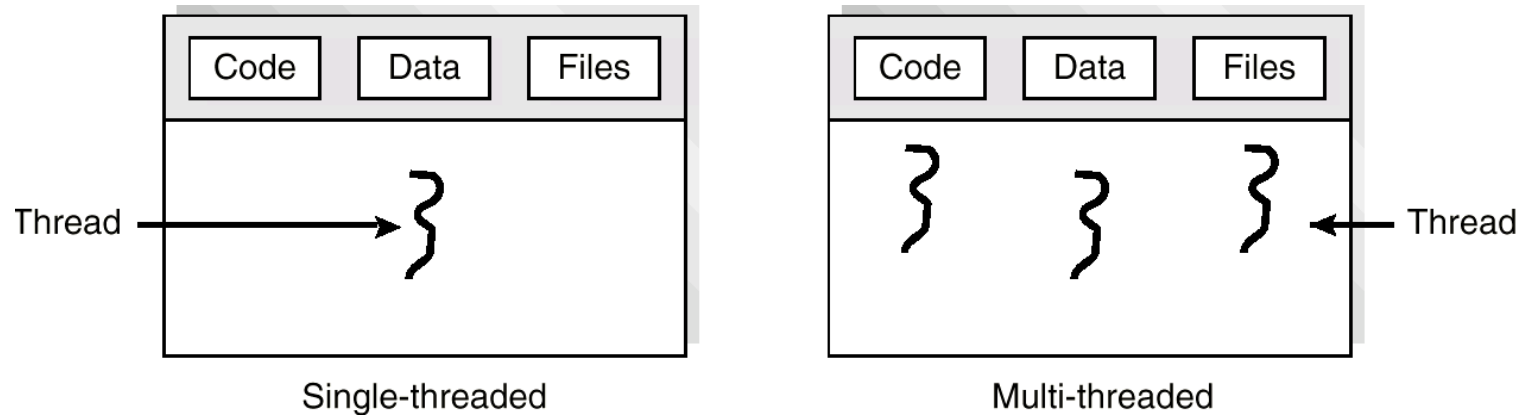
Review on Threads

- Benefits
- C Threads
- Java Threads

Benefits

- Responsiveness
- Resource Sharing
- Economy
- Utilization of MP Architectures

Single and Multithreaded Processes



C Threads

- **Primitives**

- **Creation**

- *int pthread_create(pthread_t*thread, const pthread_attr_t*attr, void* (*start_routine)(void*), void*arg);*

- **Synchronization:**

- *int pthread_join(pthread_t thread, void**value_ptr);*

- *suspends execution of the calling thread until the target thread terminates*

- **Termination**

- *int pthread_exit(void**value_ptr);*

C Threads

```
#include<stdio.h>
#include<pthread.h>
void f_thread(int *i) {
    printf("This is thread number %d of Process %d of ID %d\n", *i, getpid(),
    pthread_self());
}
int main(){
    int i;
    pthread_t pthread_id[3];
    for (i=0; i<3; i++)
        if (pthread_create(pthread_id+ i, NULL, f_thread, &i) == -1)
            fprintf(stderr, "erreur de creationpthreadnumero%d\n", i);
    printf("This is the Main Thread of Process %d with ID%d\n", getpid(),
    pthread_self());
    pthread_join(pthread_id[0], NULL);
}
```

Java Threads

- **Java Threads May be Created by:**
 - **Extending Thread class**
 - **Implementing the Runnable interface**

Extending the Thread Class

```
class Worker1 extends Thread
{
    public void run() {
        System.out.println("I am a Worker Thread");
    }
}
```


Creating the Thread

```
public class First
{
    public static void main(String args[]) {
        Worker1 runner = new Worker1();

        runner.start();

        System.out.println("I am the main thread");
    }
}
```

The Runnable Interface

```
public interface Runnable  
{  
    public abstract void run();  
}
```

Implementing the Runnable Interface

```
class Worker2 implements Runnable
{
    public void run() {
        System.out.println("I am a Worker Thread");
    }
}
```

Creating the Thread

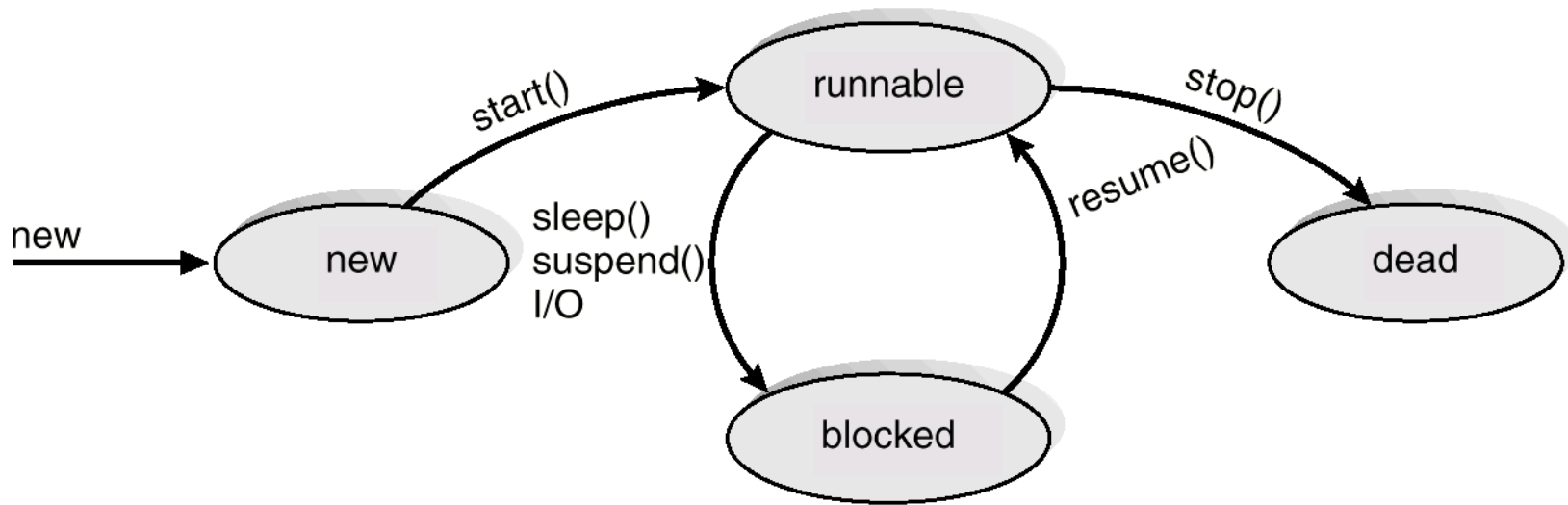
```
public class Second
{
    public static void main(String args[]) {
        Runnable runner = new Worker2();
        Thread thrd = new Thread(runner);
        thrd.start();

        System.out.println("I am the main thread");
    }
}
```

Java Thread Management

- **suspend()** – suspends execution of the currently running thread.
- **sleep()** – puts the currently running thread to sleep for a specified amount of time.
- **resume()** – resumes execution of a suspended thread.
- **stop()** – stops execution of a thread.

Java Thread States

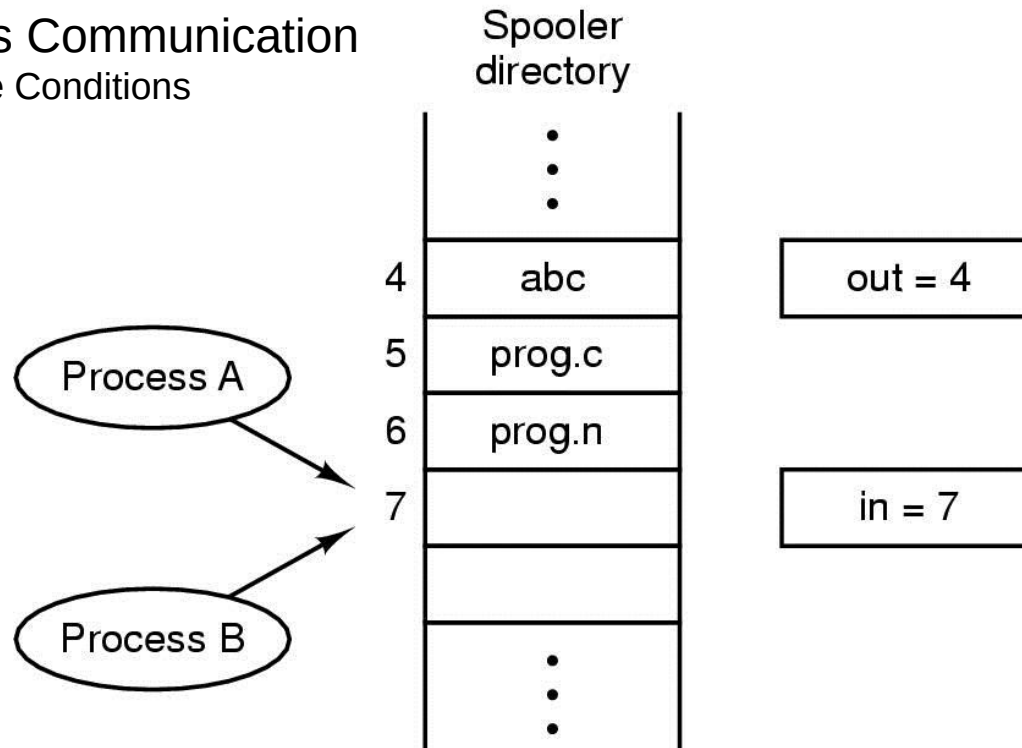


Synchronization Background

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

Background – Cont'd

Interprocess Communication
Race Conditions



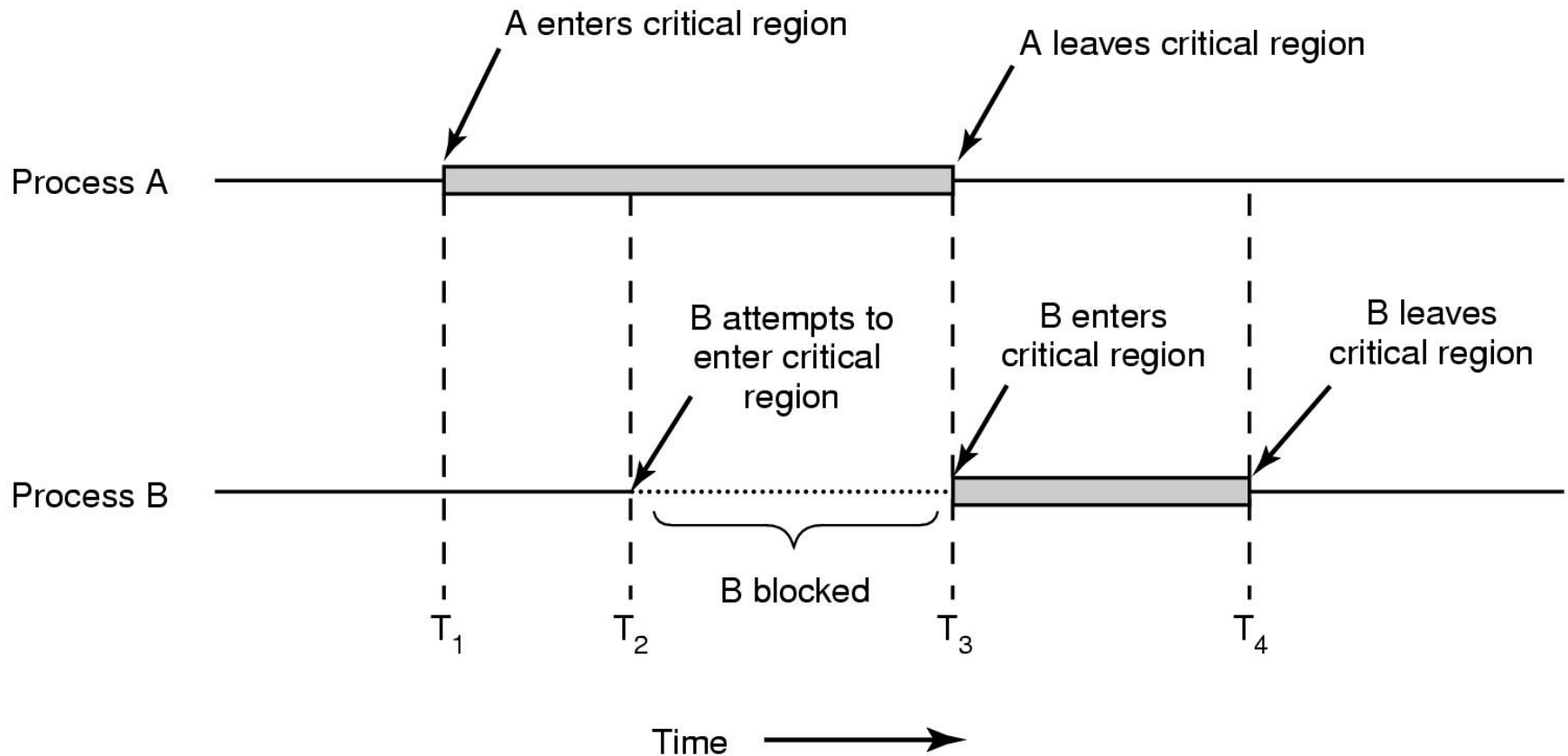
Two processes want to access shared memory at same time

Critical Section problem

Four conditions to provide mutual exclusion

1. No two processes simultaneously in critical region
2. No assumptions made about speeds or numbers of CPUs
3. No process running outside its critical region may block another process
4. No process must wait forever to enter its critical region

Critical Section problem – cont'd



Mutual exclusion using critical regions

Java Synchronization

- to control the access of multiple threads to any shared resource
- Synchronized, wait(), notify() statements
- Multiple Notifications
- Method/Block Synchronization
 - Synchronized block can be used to perform synchronization on any specific resource of the method
 - Static Synchronization
 - ❄ *If you make any static method as synchronized, the lock will be on the class not on object*

Key Differences

Feature	Method Synchronization	Static Synchronization
Applies to	Instance method	Static method
Lock object	this (current object)	ClassName.class
Lock scope	Per object	Per class
Multiple objects	Each has separate lock	One shared lock
Concurrency allowed?	Yes (if different obj) ↓	No (across all objects)

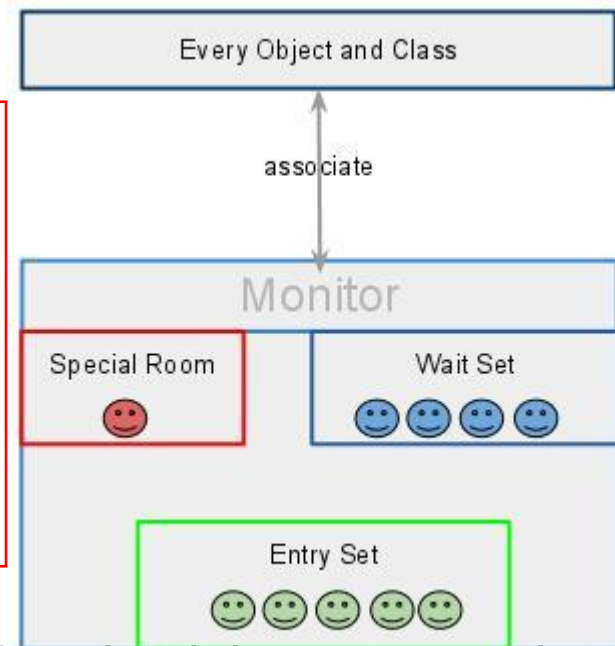
- Java Monitors
- Java Semaphores

Java Monitor

- used in Java synchronization
- If a customer wants to occupy the special room, he has to enter the Hallway(Entry Set) to wait first. Scheduler will pick one based on some criteria(e.g. FIFO).
- Implemented as binary semaphore

>Allows only one thread at a time to execute a critical section
>Provides a way for threads to wait and notify each other
>In Java, every object has an associated monitor.

+Every object has one monitor
+Only one thread can hold a monitor at a time
+Used automatically by synchronized
+Enables inter-thread communication



A Java monitor is the mechanism behind synchronized that controls thread access to an object and supports thread communication using wait() and notify().

synchronized Statement

- Every object has a lock associated with it.
- Calling a synchronized method requires “owning” the lock.
- If a calling thread does not own the lock (another thread already owns it), the calling thread is placed in the wait set for the object’s lock.
- The lock is released when a thread exits the synchronized method.

Synchronized block – Example

```
public class ThreadA {  
    public static void main(String[] args){  
        ThreadB b = new ThreadB();  
        b.start();  
        synchronized(b){  
            try{  
                System.out.println("Waiting for b to complete...");  
                b.wait();  
            }catch(InterruptedException e){  
                e.printStackTrace();  
            }  
            System.out.println("Total is: " + b.total);  
        }  
    }  
}
```

Instead of synchronizing an entire method, Java allows synchronization on specific blocks of code. This improves performance by locking only the necessary section.

Synchronized Block – Example Cont'd

```
class ThreadB extends Thread{  
    int total;  
    @Override  
    public void run(){  
        synchronized(this){  
            for(int i=0; i<100 ; i++){  
                total += i;  
            }  
            notify();  
        }  
    }  
}
```

Synchronized Method – Example

```
class Table{  
    synchronized void printTable(int n){//synchronized method  
        for(int i=1;i<=5;i++){  
            System.out.println(n*i);  
            try{  
                Thread.sleep(400);  
            }catch(Exception e){System.out.println(e);}  
        }  
    }  
}
```

A synchronized method ensures that only one thread can execute it at a time on the same object instance

Synchronized Method – Example Cont'd

```
class MyThread1 extends Thread{  
    Table t;  
    MyThread1(Table t){  
        this.t=t;  
    }  
    public void run(){  
        t.printTable(5);  
    }  
}
```

```
class MyThread2 extends Thread{  
    Table t;  
    MyThread2(Table t){  
        this.t=t;  
    }  
    public void run(){  
        t.printTable(100);  
    }  
}
```

Synchronized Method – Example Cont'd

```
public class TestSynchronization2{  
    public static void main(String args[]){  
        Table obj = new Table();//only one object  
        MyThread1 t1=new MyThread1(obj);  
        MyThread2 t2=new MyThread2(obj);  
        t1.start();  
        t2.start();  
    }  
}
```

If two threads access :
>Same object
>Multiple synchronized
instance methods

☞ Only one thread runs at a time. Because there is only one object lock.

Example : What is the difference?

```
class A {  
    public synchronized void methodA() {  
        //method A  
    }  
  
    public synchronized void methodB() {  
        // method B  
    }  
}
```

```
class B {  
    private final Object lockA = new Object();  
    private final Object lockB = new Object();  
    public void methodA() {  
        synchronized(lockA) {  
            //method A  
        }  
    }  
    public void methodB() {  
        synchronized(lockB) {  
            //method B  
        }  
    }  
}
```

Semaphore

- Synchronization tool that does not require busy waiting.
- Semaphore S – integer variable
- can only be accessed via two indivisible (atomic) operations

P or down or wait: entering

V or up or signal or notify: leaving

Semaphore as General Synchronization Tool

Semaphore S; // initialized to 1

P(S);

CriticalSection()

V(S);

Semaphore Eliminating Busy-Waiting

```
P(S) {  
    value--;  
    if (value < 0) {  
        add this process to list  
        block  
    }  
}  
  
V(S) {  
    value++;  
    if (value <= 0) {  
        remove a process P from list  
        wakeup(P);  
    }  
}
```

Deadlock and Starvation

- Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes.
- Let S and Q be two semaphores initialized to 1

P_0	P_1
P(S);	P(Q);
P(Q);	P(S);
⋮	⋮
V(S);	V(Q);
V(Q)	V(S);

- Starvation – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.

Two Types of Semaphores

- *Counting* semaphore – integer value can range over an unrestricted domain.
- *Binary* semaphore – integer value can range only between 0 and 1; for mutual exclusion

Java Semaphore

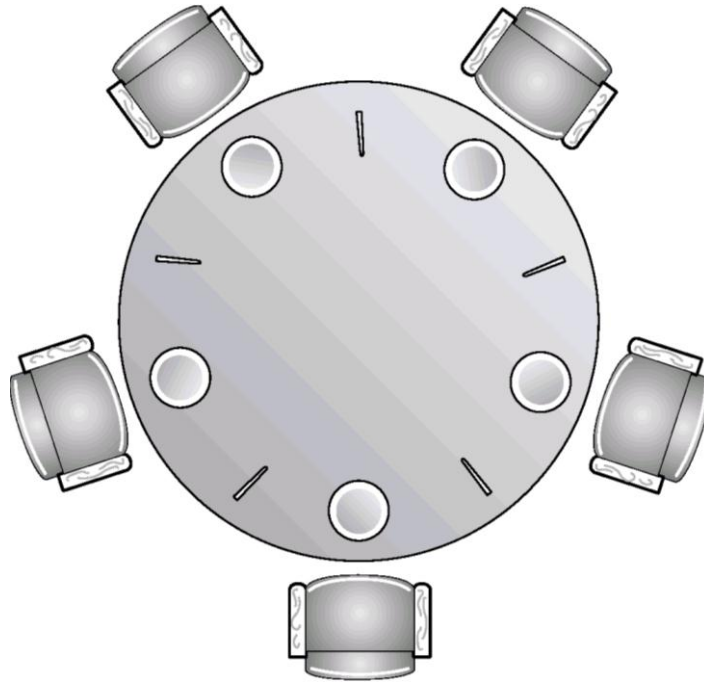
- <https://developer.android.com/reference/java/util/concurrent/Semaphore.html>

public class Semaphore extends Object implements Serializable
java.lang.Object

↳ java.util.concurrent.Semaphore

- Basic Constructor Semaphore(int permits)
 - Creates a Semaphore with the given number of permits and nonfair fairness setting.
- Fundamental methods: acquire and release
 - void acquire(): Acquires a permit from this semaphore, blocking until one is available, or the thread is interrupted.
 - void release(): Releases a permit, returning it to the semaphore.

Dining-Philosophers Problem



- Shared data

```
Semaphore fork[] = new Semaphore[5];
```

Dining-Philosophers Problem (Cont.)

- Philosopher i :

```
while (true) {  
    // think for awhile  
    // get left chopstick  
    fork[i].P();  
    // get right chopstick  
    fork[(i + 1) % 5].P();  
  
    // eat for awhile  
  
    //return left chopstick  
    fork[i].V();  
    // return right chopstick  
    fork[(i + 1) % 5].V();  
}
```

Review the algo

```
#define N 5                                     /* number of philosophers */

void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                               /* philosopher is thinking */
        take_fork(i);                          /* take left fork */
        take_fork((i+1) % N);                  /* take right fork; % is modulo operator */
        eat( );                                /* yum-yum, spaghetti */
        put_fork(i);                           /* put left fork back on the table */
        put_fork((i+1) % N);                   /* put right fork back on the table */
    }
}
```

A nonsolution to the dining philosophers problem : why?

Dining Philosophers – a solution

```
#define N          5          /* number of philosophers */
#define LEFT      (i+N-1)%N   /* number of i's left neighbor */
#define RIGHT     (i+1)%N     /* number of i's right neighbor */
#define THINKING  0          /* philosopher is thinking */
#define HUNGRY    1          /* philosopher is trying to get forks */
#define EATING    2          /* philosopher is eating */
typedef int semaphore;        /* semaphores are a special kind of int */
int state[N];                /* array to keep track of everyone's state */
semaphore mutex = 1;         /* mutual exclusion for critical regions */
semaphore s[N];              /* one semaphore per philosopher */

void philosopher(int i)      /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {           /* repeat forever */
        think( );            /* philosopher is thinking */
        take_forks(i);        /* acquire two forks or block */
        eat( );               /* yum-yum, spaghetti */
        put_forks(i);         /* put both forks back on table */
    }
}
```

Dining Philosophers

```
void take_forks(int i)                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                      /* enter critical region */
    state[i] = HUNGRY;                 /* record fact that philosopher i is hungry */
    test(i);                           /* try to acquire 2 forks */
    up(&mutex);                         /* exit critical region */
    down(&s[i]);                        /* block if forks were not acquired */
}

void put_forks(i)                     /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                      /* enter critical region */
    state[i] = THINKING;              /* philosopher has finished eating */
    test(LEFT);                       /* see if left neighbor can now eat */
    test(RIGHT);                      /* see if right neighbor can now eat */
    up(&mutex);                        /* exit critical region */
}

void test(i)                          /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Solution to dining philosophers problem (part 2)